

patX: Interpretable Pattern Extraction for Time Series and Spatial Data

First Author¹, Second Author², Third Author¹

¹Institution One

²Institution Two

September 18, 2025

Abstract

We introduce **patX**, an open-source Python package that automatically discovers compact, highly predictive, and interpretable patterns from biomedical time-series and spatial data. By framing pattern discovery as a constrained optimization problem over a pool of candidate subsequences, patX learns human-readable patterns that serve as transparent input features for conventional machine-learning models. We demonstrate patX on four diverse biomedical tasks: (i) glucose forecasting (AZT1D), (ii) arrhythmia classification (MITBIH), (iii) seizure classification (Bonn EEG), and (iv) gene expression prediction from epigenetic signatures (REMC). Across four of the five tasks, patX outperforms tsfresh and convolutional neural networks, while providing direct interpretability and significant speed-ups at inference time. We also demonstrate the interpretability of patX patterns by visualizing pattern and the area at which they are active. Our tool is available at <https://github.com/Prgrmmrjns/patX> and offers a simple interface for extracting patterns from time series and spatial data.

1 Introduction

Biomedical research increasingly relies on high-resolution time-series and spatial assays, from electrocardiography and electroencephalography to epigenomic profiling and electronic health records. These data enable clinically relevant tasks such as arrhythmia and seizure detection, gene expression prediction from chromatin marks, and risk stratification from multivariate vitals. However, extracting informative and interpretable features that translate across cohorts remains challenging.

Two dominant paradigms have emerged. Feature engineering libraries summarize signals via large collections of statistical and spectral descriptors (e.g., tsfresh [?]), often yielding strong baselines but thousands of features that can be slow to compute and hard to interpret at the instance level. Deep neural networks, including convolutional models, can learn powerful representations end-to-end but are frequently data-hungry and difficult to explain in high-stakes settings. There is a growing need for methods that marry the predictive strength of automatic feature learning with the transparency of human-readable representations. In prior work, we explored pattern-based feature extraction for gene expression prediction using swarms to optimize interpretable motifs [?]. Here, we generalize and evolve these ideas into a unified framework for time series and spatial signals.

We address this gap with **patX**, an approach that discovers a small set of human-interpretable patterns directly from data. patX searches over start/end indices, channels, and polynomial shape parameters to propose candidate motifs, each of which induces a similarity feature based on root mean squared error to the corresponding signal segment. By evaluating utility with a lightweight

learner and refining candidates via guided search, patX builds compact feature sets that are fast at inference and easy to inspect.

We evaluate patX across heterogeneous biomedical problems: arrhythmia classification on MIT-BIH [?, ?], seizure classification on Bonn EEG [?], gene expression prediction from histone modification profiles (REMC) [?], and clinical prediction on MIMIC-IV [?], as well as a glucose forecasting task (AZT1D). We compare against tsfresh [?] and a convolutional neural network baseline under matched splits and metrics.

Our contributions are:

- We introduce patX, a dynamic polynomial pattern search that produces human-readable motifs and associated similarity features.
- We demonstrate strong performance with few features, favorable runtime, and direct interpretability across diverse tasks and data modalities.
- We release patX as an official Python package on PyPI and actively maintain it (link: <https://pypi.org/project/patx/>).
- We provide a simple, reproducible pipeline and visual tools that overlay learned motifs and highlight activation regions for qualitative assessment.

The package is available at <https://github.com/Prgrmmrjns/patX>.

2 Methods

2.1 Pattern Definition

Given a univariate or multivariate signal $\mathbf{x} \in \mathbb{R}^{T \times C}$ with length T and C channels, a *pattern* p is a polynomial template over a contiguous interval $[s, e]$ with $L = e - s + 1$. We z-score inputs per channel (or per subject where appropriate). The primary similarity feature is the RMSE between $\mathbf{x}$ restricted to $[s, e]$ and p ; we optionally compute robust variants (e.g., MAE) during ablations. Polynomial degree defaults to 3 but is tunable; multi-channel data attach a channel identifier to each pattern.

2.2 Optimization Framework

We search over $(s, e, \mathbf{c})$ where $\mathbf{c}$ are polynomial coefficients defining p , and, for multivariate data, a channel index. Each candidate induces one feature: $\text{RMSE}(\mathbf{x}_{[s:e]}, p(\mathbf{c}))$. At each round, Optuna proposes a batch of candidates guided by validation loss from a LightGBM model with early stopping; we append the best candidate if it improves the held-out score. We cache segment features to avoid recomputation and apply backward elimination to prune redundant patterns. Typical budgets are 50–200 rounds with 50–200 proposals per round, using 6 CPU threads. We target small final sets (≤ 12 motifs) to minimize latency and ease interpretation.

patX is available as an official Python package and is actively maintained on PyPI (link: <https://pypi.org/project/patx/>).

2.3 patX Optimization Algorithm

We outline the iterative search procedure. Candidate patterns are encoded as start–end indices (and channel id for multi-series data). LightGBM supplies the validation loss to Optuna, which selects the next candidate batch via Tree-Parzen Estimator.

The implementation centers on the `PatternExtractor` class. Inputs are converted to `float32` numpy arrays; for multi-series inputs we stack series along axis 1, yielding shape $N \times S \times T$. For a proposed pattern with start s , width w , and polynomial coefficients $\{c_i\}_{i=0}^d$, we generate a template over $[-1, 1]$ using a dense grid of w points and evaluate the polynomial to obtain values $p \in \mathbb{R}^w$. The feature for a sample is the RMSE between its segment $x_{[s:s+w]}$ and p . We maintain a cached design matrix that concatenates any provided initial features (e.g., raw glucose and time) with newly accepted pattern features to avoid recomputation.

At each round, Optuna optimizes over s , w , coefficients, and (optionally) channel index to maximize validation performance (or minimize RMSE). A candidate is accepted if it improves the best held-out score; otherwise the search halts after no improvement. We retrain the downstream model on the final feature matrix, and we construct a test matrix by combining test initial features (if provided) and per-pattern features computed on test segments. For multi-series inputs, per-pattern features are computed from the selected channel. The optimizer exports learned patterns (start, width, coefficients, and channel id) to JSON for reproducibility and downstream use.

2.4 Data Pre-processing

AZT1D (glucose forecasting). We parse CGM records per subject and create supervised learning examples using lagged glucose features. Specifically, we construct 24 lag variables $\{g_{t-\ell}\}_{\ell=0}^{23}$ and define the regression target as the h -step change $g_{t+h} - g_t$ (where h is the prediction horizon). We drop rows with missing values induced by shifts, then split into train/test with a fixed random seed. In addition, we pass two initial features to patX (raw glucose and time-of-day) so that patterns can augment a simple contextual baseline. No scaling is applied beyond NaN filtering. Link: <https://data.mendeley.com/datasets/gk9m674wcx/1>.

MITBIH (arrhythmia classification). We use a binned feature matrix with a target column. We stratify a 75/25 train/test split. Labels are integer-encoded. This dataset originates from the PhysioNet MIT-BIH Arrhythmia Database (link: <https://www.kaggle.com/datasets/mondejar/mitbih-database>).

Bonn EEG (five-class EEG). We load a preprocessed table with segments. We stratify train/test splits and work with dense `float32` matrices. Link: <https://dataverse.csuc.cat/dataset.xhtml?persistentId=doi:10.34810/data490>.

REMC (ChIP-seq). For each cell line, we load a feature table with a binary target and multiple histone mark tracks. We form multi-series inputs by grouping columns by mark prefix (e.g., promoter/enhancer-associated marks), then stratify train/test splits. patX searches over start/width/degree per mark (channel) to learn a small set of interpretable patterns. Performance is measured via AUC. Roadmap Epigenomics portal: https://egg2.wustl.edu/roadmap/web_portal/processed_data.html. Link of processed data: <https://gitlab.gwdg.de/MedBioinf/generegulation/patternchrome>

MIMIC-IV (EHR classification). We preprocess MIMIC-IV to derive one row per subject with 24 hourly samples per time series (e.g., heart rate, respiratory rate, blood pressure, labs) and a binary ARDS flag. Preprocessing includes: cleaning required columns, sorting by subject and hour, replacing infinities, capping extremes at the 0.1/99.9th percentiles, forward/backward filling within subjects, and padding/truncating to 24 hours per series. We rename series to clean identifiers (e.g., hourly heart rate 0.23). We then group columns by series name to create a multi-series input list, standardize each series on the train split, and provide age as an initial feature (standardized) to patX. We use stratified splits and report accuracy. Link: <https://physionet.org/content/mimiciv/3.1/>.

2.5 Datasets

- **AZT1D**: CGM and insulin pump data for type 1 diabetes forecasting (15/30 min horizons)
- **MITBIH**: ECG rhythm classification using the MIT-BIH Arrhythmia Database
- **Bonn EEG**: Five-class EEG dataset (Z,O,N,F,S) used in seizure research
- **REMC**: Histone modification profiles with gene expression labels from Roadmap Epigenomics
- **MIMIC-IV**: EHR time series for clinical prediction tasks

2.6 Baselines

We compare against:

1. **tsfresh + LightGBM**: tsfresh features with LightGBM
2. **CNN**: A PyTorch 1D CNN baseline with early stopping; we use the same train/validation splits, and dataset-appropriate metrics (accuracy/AUC/RMSE). The network ingests fixed-length vectors and outputs class probabilities or regression predictions.

All baselines share the same data splits and evaluation protocol for a fair comparison.

2.6.1 TSFRESH Baseline Implementation

For the tsfresh baseline, we use `MinimalFCParameters` to extract a compact descriptor set including moments, spectral features, and shape statistics. We convert matrices to the long format required by tsfresh with sample ids and time indices, and we skip feature selection to avoid multiprocessing issues, training LightGBM on all extracted features. Hyperparameters and evaluation are kept identical to patX and other baselines.

2.7 Evaluation Metrics

Classification uses area under the ROC curve (AUC) or accuracy; regression uses root mean squared error (RMSE). We also report the number of extracted patterns and wall-clock extraction time. Where applicable, we compute 95% CIs across seeds.

2.8 Experimental Protocol

We split data 75%/25% train/test with fixed seeds and preserve class balance. Pattern search uses only training data. LightGBM is tuned via 5-fold CV on training features and retrained on the full train split. Results are averaged across 3 seeds. We fix the total pattern budget to ensure comparable runtime across datasets and perform ablations on degree, window length, and similarity.

3 Results

3.1 Performance Comparison

Table 1 presents the test set performance of patX + LightGBM, tsfresh + LightGBM, and CNN baselines across five biomedical datasets. We report mean and standard deviation.

Table 1: Performance comparison across biomedical datasets showing mean $\pm$ standard deviation across folds/subjects.

Dataset	Method	Performance	# Features	Time (s)
AZT1D	PATX	18.877 $\pm$ 0.531	4.3 $\pm$ 0.6	29.7 $\pm$ 2.0
	TSFRESH	20.010 $\pm$ 0.812	10.0 $\pm$ 0.0	1.8 $\pm$ 0.2
	CNN	21.931 $\pm$ 1.116	24.0 $\pm$ 0.0	33.0 $\pm$ 3.7
MITBIH	PATX	0.879 $\pm$ 0.007	11.2 $\pm$ 2.6	30.6 $\pm$ 7.1
	TSFRESH	0.817 $\pm$ 0.003	10.0 $\pm$ 0.0	1.9 $\pm$ 0.0
	CNN	0.678 $\pm$ 0.033	100.0 $\pm$ 0.0	77.3 $\pm$ 1.2
Bonn EEG	PATX	0.744 $\pm$ 0.015	4.0 $\pm$ 1.2	38.5 $\pm$ 10.5
	TSFRESH	0.584 $\pm$ 0.036	10.0 $\pm$ 0.0	0.7 $\pm$ 0.1
	CNN	0.422 $\pm$ 0.126	4101.0 $\pm$ 0.0	153.2 $\pm$ 73.3
REMC	PATX	0.925 $\pm$ 0.010	8.6 $\pm$ 2.3	170.0 $\pm$ 166.4
	TSFRESH	0.876 $\pm$ 0.009	10.0 $\pm$ 0.0	15.9 $\pm$ 43.8
	CNN	0.871 $\pm$ 0.009	1000.0 $\pm$ 0.0	3927.3 $\pm$ 4543.6
MIMIC-IV	PATX	0.777 $\pm$ 0.006	6.2 $\pm$ 2.3	57.2 $\pm$ 48.0
	TSFRESH	0.769 $\pm$ 0.004	10.0 $\pm$ 0.0	1.2 $\pm$ 0.1
	CNN	0.747 $\pm$ 0.005	697.0 $\pm$ 0.0	287.1 $\pm$ 3.6

3.2 Pattern Discovery Characteristics

3.3 Interpretability Case Studies

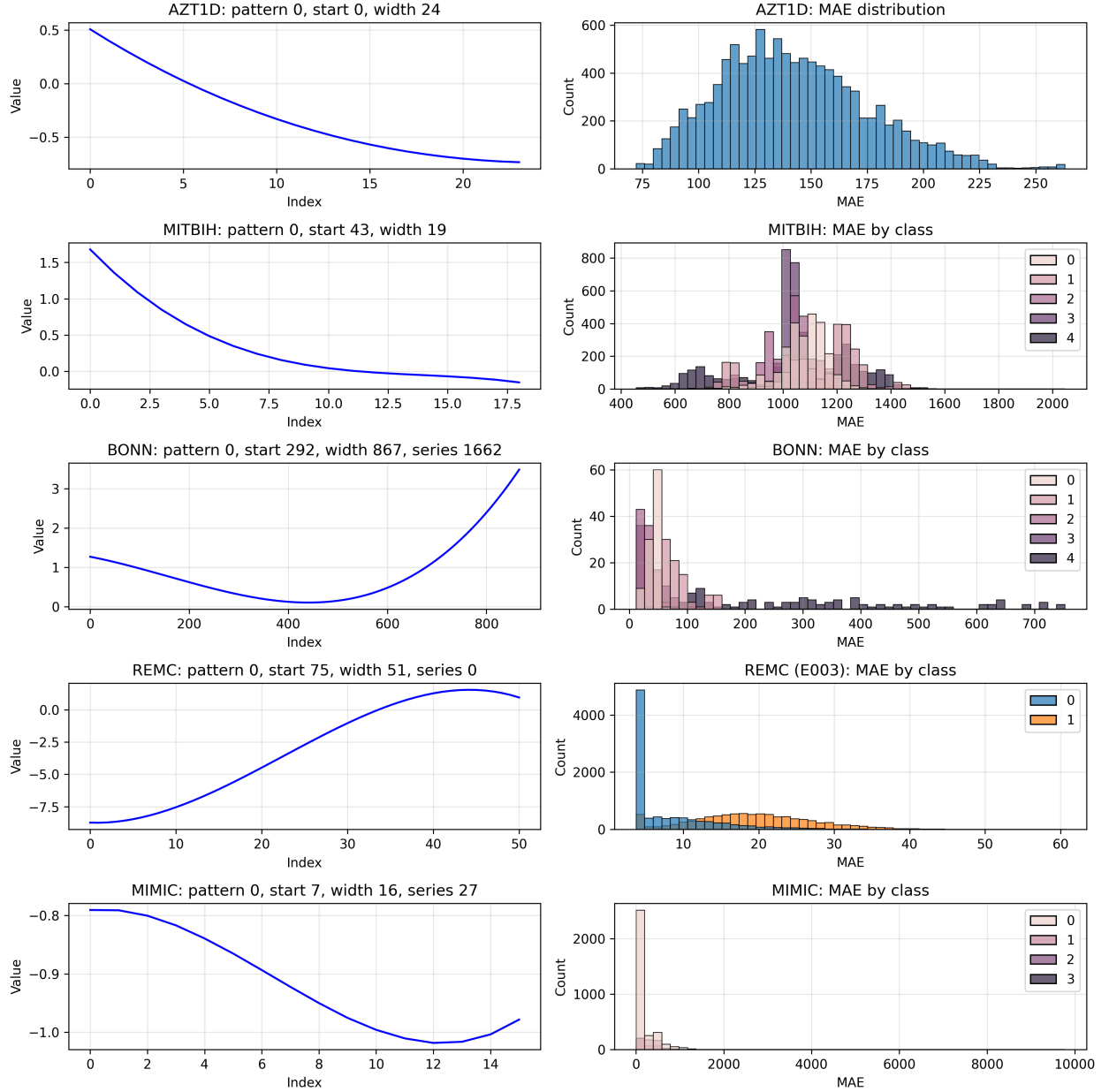


Figure 1: First discovered pattern per dataset (left: template shape; right: value distribution). For REMC and AZT1D, we use the first pattern of the first sample.

3.4 Visualizing search space and decision boundaries

3.5 Ablation Study

4 Discussion

5 Conclusion

We introduced patX, an interpretable pattern extraction framework bridging the gap between hand-crafted descriptors and opaque deep models. Through extensive experiments, we demonstrated that a small set of automatically discovered motifs can match or exceed the predictive power of deep learning across heterogeneous biomedical domains, while offering orders-of-magnitude reductions in feature count and inference cost. Future extensions will explore adaptive pattern lengths, semi-supervised pretraining on large unlabeled corpora, and tighter integration with interactive visualization tools to facilitate clinical adoption.